SQL Triggers

A trigger in SQL is a set of procedural statements that automatically execute in response to certain events on a specified table or view. Triggers help automate tasks, enforce business rules, maintain data integrity, and log changes in the database.

Types of Triggers

1. **Before Triggers**
   Execute before the triggering event occurs. Commonly used for validation or data modification before making changes to the database.

   Ex: BEFORE INSERT, BEFORE UPDATE, BEFORE DELETE

2. **After Triggers**
   Execute after the triggering event occurs. Commonly used for logging changes or performing calculations after data is modified.

   Ex: AFTER INSERT, AFTER UPDATE, AFTER DELETE

3. **Instead of Triggers**
   Override the default action of a triggering event, typically used with views.

   Ex: INSTEAD OF INSERT, INSTEAD OF UPDATE, INSTEAD OF DELETE

Components of a Trigger

Event
The database operation (INSERT, UPDATE, DELETE) that activates the trigger.
Condition
Optional logic specifying when the trigger should execute.
Action
The SQL statements that define the operation to be performed.

The primary features that set triggers apart from stored procedures are as follows:

i). Triggers cannot be manually executed or invoked.
ii). There is no possibility that triggers will receive parameters.
iii). Within a trigger, a transaction cannot be committed or rolled back

Syntax

```
CREATE TRIGGER trigger_name

{ BEFORE | AFTER | INSTEAD OF }

{ INSERT | UPDATE | DELETE }

ON table_name

[ FOR EACH ROW | FOR EACH STATEMENT ]

[ WHEN (condition) ]

BEGIN

  -- SQL statements

END;
```

Within the trigger body, the `OLD` and `NEW` keywords enable you to access columns in the rows affected by a trigger. `OLD` and `NEW` are MySQL extensions to triggers; they are not case-sensitive.

In an `INSERT` trigger, only `NEW.`*`col_name`* can be used; there is no old row. In a `DELETE` trigger, only `OLD.`*`col_name`* can be used; there is no new row. In an `UPDATE` trigger, you

can use OLD.*col_name* to refer to the columns of a row before it is updated and NEW.*col_name* to refer to the columns of the row after it is updated.

A column named with OLD is read only. You can refer to it (if you have the SELECT privilege), but not modify it. You can refer to a column named with NEW if you have the SELECT privilege for it. In a BEFORE trigger, you can also change its value with SET NEW.*col_name* = *value* if you have the UPDATE privilege for it. This means you can use a trigger to modify the values to be inserted into a new row or used to update a row. (Such a SET statement has no effect in an AFTER trigger because the row change has already occurred.)

By using the BEGIN ... END construct, you can define a trigger that executes multiple statements. Within the BEGIN block, you also can use other syntax that is permitted within stored routines such as conditionals and loops. However, just as for stored routines, if you use the **mysql** program to define a trigger that executes multiple statements, it is necessary to redefine the **mysql** statement delimiter so that you can use the ; statement delimiter within the trigger definition. The following example illustrates these points. It defines an UPDATE trigger that checks the new value to be used for updating each row, and modifies the value to be within the range from 0 to 100. This must be a BEFORE trigger because the value must be checked before it is used to update the row:

```
delimiter //
CREATE TRIGGER upd_check
BEFORE UPDATE
ON account
FOR EACH ROW
    BEGIN
      IF NEW.amount < 0 THEN
        SET NEW.amount = 0;
      ELSEIF NEW.amount > 100 THEN
        SET NEW.amount = 100;
      END IF;
```

```
    END;//
delimiter ;
```

MySQL handles errors during trigger execution as follows:

- If a `BEFORE` trigger fails, the operation on the corresponding row is not performed.
- A `BEFORE` trigger is activated by the *attempt* to insert or modify the row, regardless of whether the attempt subsequently succeeds.
- An `AFTER` trigger is executed only if any `BEFORE` triggers and the row operation execute successfully.
- An error during either a `BEFORE` or `AFTER` trigger results in failure of the entire statement that caused trigger invocation.
- For transactional tables, failure of a statement should cause rollback of all changes performed by the statement. Failure of a trigger causes the statement to fail, so trigger failure also causes rollback. For nontransactional tables, such rollback cannot be done, so although the statement fails, any changes performed prior to the point of the error remain in effect.

It can be easier to define a stored procedure separately and then invoke it from the trigger using a simple `CALL` statement. This is also advantageous if you want to execute the same code from within several triggers.

There are limitations on what can appear in statements that a trigger executes when activated:

- The trigger cannot use the `CALL` statement to invoke stored procedures that return data to the client or that use dynamic SQL. (Stored procedures are permitted to return data to the trigger through `OUT` or `INOUT` parameters.)
- The trigger cannot use statements that explicitly or implicitly begin or end a transaction, such as `START TRANSACTION`, `COMMIT`, or `ROLLBACK`. (`ROLLBACK to SAVEPOINT` is permitted because it does not end a transaction.).

To list all the triggers you have created: SHOW TRIGGERS;

Task to do

```
mysql> select * from employee;
+----+------+---------+--------+------------+------+
| id | name | superid | salary | bdate      | dno  |
+----+------+---------+--------+------------+------+
|  1 | john |       3 | 100000 | 1960-01-01 |    1 |
|  2 | mary |       3 |  50000 | 1964-12-01 |    3 |
|  3 | bob  |    NULL |  80000 | 1974-02-07 |    3 |
|  4 | tom  |       1 |  50000 | 1970-01-17 |    2 |
|  5 | bill |    NULL |   NULL | 1985-01-20 |    1 |
+----+------+---------+--------+------------+------+
5 rows in set (0.00 sec)

mysql> select * from deptsal;
+---------+-------------+
| dnumber | totalsalary |
+---------+-------------+
|       1 |      100000 |
|       2 |       50000 |
|       3 |      130000 |
+---------+-------------+
3 rows in set (0.00 sec)
```

1. From the tables above, we want to create a trigger to update the total salary of a department when a new employee is hired
2. A trigger to update the total salary of a department when an employee tuple is modified
3. A trigger to update the total salary of a department when an employee tuple is deleted